ABRA — Technical Documentation

Version 3.29.0 · Last updated 2026-07-31

Written in ASD-STE100 Simplified Technical English. Sentences are short. The voice is active. One word has one meaning. The document follows the Diátaxis structure: Tutorial, How-to, Reference, Explanation.

---

1. Tutorial — run ABRA for the first time

Do these steps in order.

1. Get the code. Clone the repository `github.com/willhoop/abra`.
2. Get the sibling engine. Clone `github.com/willhoop/chomp` next to it. ABRA reads it at `../CHOMP`.
3. Open the site. Open `web/index.html` in a web browser. The site needs no build step and no server.
4. Visit a model. Click a house in the town. Open **PORY** and move the sliders. The win % updates.
5. Run one check. In a terminal, run `node engine/validate_damage.js`. It confirms the damage engine.

You now have the site and one validated model.

2. How-to — common tasks

Repair the raw archive before any reparse. `MODE=backfill node engine/durable-ingest.js data/games.ladder.jsonl` The hourly Action appends to the store while the raw archive is gitignored, so CI-collected games have no local raw log. `MODE=reparse` REFUSES to run while any stored game is missing one, because reparse rebuilds the store from the archive and would delete them.

Run the official Champions engine. `SHOWDOWN_PATH=/path/to/pokemon-showdown node engine/champions_sim.js` Needs a BUILT master checkout; the champions mod is not in the npm package.

Generate self-play games (MEW). `SHOWDOWN_PATH=... node engine/mew.js --n 1000` then `SHOWDOWN_PATH=... node engine/validate_selfplay.js` Output goes to `data/games.selfplay.jsonl` and must NEVER be pooled with the ladder store.

Fit the scoring bot's policy. `SHOWDOWN_PATH=... node engine/fit_policy.js` Reads every clean open-team-sheet game, builds one row per human decision, fits by conditional logit, and writes `data/policy-weights.json`. It prints the held-out comparison against the behaviour clone; if the fit does not beat the clone it says so in plain words and the weights must not ship. `engine/selftest.js` refuses a weight file that lost to the clone, and refuses one whose feature list does not match what `engine/board.js` computes.

Check the corpus the policy was fitted on. `SHOWDOWN_PATH=... node engine/corpus_shift.js` Compares open-sheet against closed-sheet ladder on the same code: team composition, behaviour given a board, bot contamination before and after filtering, and mean rating. Run it before trusting a refit. The teams differ by 551.9 points of total absolute species difference; the measured behaviour differs by at most 1.49. `fit_policy.js` then re-estimates on a sample reweighted to the closed-sheet species mix and reports the shift **in standard errors**. Five weights move materially — `priorLogP` by 10.8 SE, `bp` by 6.2 SE (its sign flips) — so the **reweighted vector ships**. Board-reading weights (`eff`, `immune`, `deadStatus`) do not move: reading the board transfers between the two metagames, how much popularity is worth does not.

Pull the Bo3 open-team-sheet ladder. `PAGES=6 CONC=20 FORMATS=gen9championsvgc2026regmbbo3 node engine/durable-ingest.js data/games.bo3.jsonl` CI does this hourly. This format's ruleset carries `Force Open Team Sheets`, so every game publishes all six sets of both sides — the only continuously-collected corpus in which the choice set of a decision is known. The main ladder format carries plain `Open Team Sheets`, which is

optional and needs both players to agree, which is why only ~1% of the closed store has sheets. **It goes in its own store and is never pooled with the ladder store** — different information regime, different metagame. Dedupe it with `python3 engine/dedupe_store.py data/games.bo3.jsonl --write`.

Play with the scoring policy. `SHOWDOWN_PATH=... node engine/mew.js --n 1000 --policy score`

`--policy`	**what it does**
`random`	Showdown's `RandomPlayerAI`. Correct for plumbing and matchup structure; not valid as training data
`prior`	samples the move a species actually clicks, from `data/move-priors.json`. Board-blind
`score`	tracks the board and scores every (move, target) pair with the fitted weights. **The only mode that aims**

Check the run's own accounting: `policy=score` must report ~100% of decisions scored and a non-zero `aiming:` line. A run reporting 0% is not a scoring bot.

Compare two policies fairly. Pass the SAME `--seed` to both runs — MEW derives its team sampling from it, so the two corpora play identical teams and the comparison is paired rather than confounded by which teams each happened to draw:

```
node engine/mew.js --n 600 --policy prior --seed 4242 --out data/_a.jsonl
node engine/mew.js --n 600 --policy score --seed 4242 --out data/_b.jsonl
node engine/realism_report.js --self data/_a.jsonl
node engine/realism_report.js --self data/_b.jsonl
```

Generate self-play at scale (the farm). `SHOWDOWN_PATH=... node engine/mew_farm.js --n 200000 --procs 12 --conc 1`

`--conc` must stay at 1. The simulator is synchronous and CPU-bound, so in-process concurrency never overlaps real work; it only holds N battles live at once and multiplies GC pressure. Measured on 8 physical / 16 logical cores: 8 procs at `--conc 4` gave 11 games/sec, the same 8 procs at `--conc 1` gave 38. Process count is the unit; 12 procs / conc 1 reproduced at 44–46 games/sec.

Two files are written, and **both are needed**:

file	**contains**	**read by**
`data/games.selfplay.jsonl`	game-level records (six / brought / lead / winner)	store-shaped engines
`data/games.selfplay.raw-logs.jsonl`	full protocol logs, `{id, uploadtime, log}`	PORY, `state_encoder.py`

The value models reconstruct per-turn board states by replaying the **protocol log**; the records are summaries and contain no per-turn state. A corpus written without the sidecar is unreadable by the models it exists to train. Budget ~5 KB per game per file.

Every battle is replayable. A record carries `id`, `selfplay.seed`, `selfplay.policy` and `selfplay.engine_commit`, and both the battle dice and both players' decision PRNGs are derived from that seed. Re-running the same seed against the same pinned engine reproduces the game byte-for-byte (verified 25/25, excluding the `|t:|` wall-clock line). This is what makes a claim like "this switch won the game" checkable rather than asserted.

Teams are validated against Showdown, not against our own rules. `packTeam` enforces Item Clause during packing, then runs the official `TeamValidator` and repairs what it can; MEW discards anything still invalid rather than recording it. `BattleStream` does **not** validate — it plays whatever it is handed — so without this gate an illegal team produces a record indistinguishable from a legitimate game. Before the gate, the validator rejected 80.5% of the pool. Cost is 4.30 ms/team, ~9.6% of a battle.

Team preview is sampled, not fixed. `chooseTeamPreview` draws four of six weighted by measured P(brought | on team) and two leads weighted by P(lead | brought), from `data/bring-priors.json` (regenerate with `node engine/bring_priors.js`). `MEW_PREVIEW_TEMP` controls how far the draw wanders from the common line: 1.0 samples proportional to measured propensity, lower collapses toward the single most likely bring, higher flattens toward uniform.

Refresh the official priors. `node engine/fetch_smogon_stats.js` then `node engine/smogon_priors.js` CI does this on the 4th and 11th of each month.

Pull new replays. `PAGES=6 CONC=20 node engine/durable-ingest.js data/games.ladder.jsonl` The command adds only new games. It never duplicates a game and never re-fetches a stored game.

Rebuild the meta model. `node engine/analyze.js data/games.ladder.jsonl` writes `data/meta-usage.json`.

Refresh the site data. `python3 engine/refresh-site-data.py` writes `data/live.js`, `data/archetypes.json`, and `data/kad-replays.js`.

Run a model or an evaluation.

- GURU matchup matrix: `python3 engine/guru.py`
- XATU belief / policy eval: `python3 engine/eval_policy.py`
- PORY value net: `python3 engine/pory.py`
- CHOMP-EV proof: `node engine/chomp_ev.js`
- SLOWKING preview-Nash (species): `python3 engine/slowking_preview.py`
- SLOWKING preview-Nash (playstyle): first `node engine/playstyle.js`, then `MATRIX_FILE=data/playstyle-matchups.json TAG=playstyle python3 engine/slowking_preview.py`

Validate the damage engine. `node engine/validate_damage.js`. It fails if any scenario is more than 5% from the Smogon damage calculator.

Run the tests. `node tests/test-parse.js`, `node tests/test-dynamics.js`, `node tests/test-medicham.js`, `node tests/test-chomp-ev.js`, `python3 tests/test-jolteon.py`, `python3 tests/test-slowking.py`.

Edit the site, then mirror it. After you change `web/index.html`, copy it: `cp web/index.html app/index.html`.

3. Reference

3.1 Stored game record (`data/games.ladder.jsonl`, one JSON object per line)

Field	Meaning
`id`, `date`	replay id and upload time
`p1`, `p2`	`{name, rating, bot}` per player
`six.p1/p2`	the six revealed at preview
`brought.p1/p2`	the four actually brought
`lead.p1/p2`	the two led
`sets`	per species, the revealed moves / item / ability
`turns`	per-turn events (move, damage, faint, status, field)
`winner`	the winning name

3.2 Model outputs

File	Written by	Contents
`data/damage-validation.json`	`validate_damage.js`	damage error against the Smogon damage calculator

File	Written by	Contents
`data/guru-matchups.json`, `guru.js`	`guru.py`	archetype matchup matrix, Wilson CIs
`data/xatu.json`, `xatu.js`	`xatu.py`	opponent set/move belief
`data/pory.js`, `pory-eval.json`	`pory.py`	mid-game value net + its score
`data/chomp-ev.json`	`chomp_ev.js`	the bring proof (null result)
`data/playstyle-matchups.json`	`playstyle.js`	playstyle matchup matrix
`data/slowking-eval.json`, `slowking-playstyle-eval.json`, `slowking*.js`	`slowking_preview.py`	preview equilibrium + exploitability
`data/policy-weights.json`	`fit_policy.js`	the scoring bot's fitted weights, the feature list they were fitted against, and the held-out comparison against the behaviour clone
`data/meta-usage.json`, `live.js`	`analyze.js`, `refresh-site-data.py`	usage model + live site counts

3.3 Continuous collection

A GitHub Action (`.github/workflows/ingest.yml`) runs the pull hourly and commits the store. A separate tests workflow runs the test suite and the damage validation on every push and pull request.

4. Explanation

4.-1 Why additivity is the recurring failure (added 3.28.0)

The same defect has now appeared at three levels of this project, and it is worth stating once because the fix has the same shape every time.

A sum of independent terms cannot express a conjunction. If a score is `w1*a + w2*b`, there is no setting of `w1` and `w2` that makes "a AND b together" worth more than the parts. Not *scored badly* — **not representable**.

Where it bites:

level	the thing that cannot be said	consequence
one Pokémon (PORY, §4.0)	"material lead matters on turn 3, not turn 25"	the neural net exists for this
two Pokémon (MAG → DODUO)	"Protect with A while B removes the thing killing A"	independent per-slot choice; MAG aims both attacks at one foe ~50% of the time where humans do 23.4%
six Pokémon (JOLTEON → DITTO)	"Pelipper + Archaludon is worth more than the sum"	measured: the additive species block moves the score by ~6.3 while the two non-additive terms move it by ~0.4, so hill-climbing converges on the top-6 by weight

The literature agrees and names the boundary. Cooperative multi-agent value factorisation (QMIX, Rashid et al. 2018) constrains the joint value to be *monotonic* in each agent's utility, and QPLEX and Weighted QMIX exist precisely because that constraint cannot represent non-monotonic coordination. Our case is the non-monotonic one.

But the expensive machinery does not apply here. That literature is about avoiding enumeration when agents are many. With **two** Pokémon the coordination graph is a single edge, so Variable Elimination degenerates to enumerating the joint actions and Max-Plus message passing is unnecessary. Measured on a real mid-game board: **9 × 8 = 72** joint actions per side, of which only **28** have a non-zero interaction term — the other 44 score exactly as the sum of two singles already computed.

And the fix is not "add the interaction term", it is "fit it for the right thing." DODUO's pair block exists and is fitted; it loses at 42.0% because it was fitted to *resemble human pairs* rather than to *win*. Expressiveness was necessary and not sufficient.

Practical rule. Before adding a feature, ask whether the thing you want to say is a conjunction. If it is, no weight on an individual term will ever say it — and if the model already has the interaction term, check what objective it was fitted to before concluding the idea failed.

4.0 What a neural network is here, and why it is not automatically an upgrade

Every model in ABRA answers one question: given the board right now, what is P(I win)? PORY answers it with **logistic regression** — multiply each feature by a weight, add them up, squash to 0..1:

p = sigmoid(w0 + w1*alive_diff* + *w2*hp_diff + ...)

That can only draw a straight dividing line through feature space. It cannot express *"being one Pokémon ahead matters enormously on turn 3 and barely on turn 25"*, because that is an **interaction** between two features, and a sum of independent terms has no way to say it.

A **neural network** is the same idea with a middle step. Inputs are combined into H hidden units, each its own weighted sum passed through a nonlinearity (ReLU: `max(0, x)`), and those are combined into the answer:

h = relu(W1 @ x + b1) # H learned intermediate quantities p = sigmoid(W2 @ h + b2)

Each hidden unit can become a detector for a **conjunction** — "ahead on material AND my active is faster AND Trick Room is not up" — and the output layer weighs the detectors. Universal approximation (Cybenko 1989; Hornik 1991) says one hidden layer of sufficient width can represent any continuous function on a bounded domain, so the network is strictly **more expressive** than the linear model.

Strictly more expressive is a claim about representation, not about learning. Extra capacity spent on features that contain no interactions buys nothing and costs variance. `engine/pory_baseline.py` already established the relevant fact: PORY's six material features are **beaten by two of them** (`alive_diff + hp_diff` at 0.5822 vs PORY's 0.5840). If the features carry no more signal, a network fitted to them lands in the same place — and reporting otherwise would be measuring the estimator rather than the game.

This is the lesson the game-playing literature learned repeatedly. **TD-Gammon** (Tesauro 1994) needed hand-designed board features, not checker counts. **AlphaGo Zero / AlphaZero** (Silver et al. 2017, 2018) feed the value head a *stack of planes* — piece positions, repetition, side to move — because material is precisely the baseline a value net must beat, and it beats it by seeing **where** the pieces are. Leela Chess Zero's ablations show the value head collapsing toward the handcrafted evaluation when the positional planes are stripped.

So the binding constraint was **representation, not capacity**. `engine/state_encoder.py` supplies the planes: HP per slot, active vs benched, status, stat boosts, weather / terrain / Trick Room / Tailwind / screens, hazards, and the active Pokémon's types — 121 features where PORY had 6.

`engine/pory_nn.py` then separates the two explanations rather than conflating them, by running eight arms on one split:

arm	what it isolates
`B2` logistic, `alive_diff + hp_diff`	**the bar** — two material features
`L6` logistic, PORY's six	PORY itself
`LR` logistic, rich features	gain from **representation** alone
`N6` network, PORY's six	gain from **nonlinearity** alone
`NR` network, rich features	both

If the network wins only on `N6`, the gain is nonlinearity. If only on `LR`, PORY was feature-starved rather than model-starved. If neither beats `B2`, that is the result and it is reported as such.

Species identity is deliberately **excluded**: ~240 species one-hot across 8 slots is a 1,920-dimensional input, which on ~9,000 real games would be fitted almost entirely to noise. Species enters only through type. Embeddings become defensible once the self-play corpus is large enough, and the encoder is versioned so that change is visible rather than silent.

Two methodological points that decide whether any of these numbers mean anything. **Splits are by game, never by state** — turns within a game share an outcome, so splitting on states leaks the label across the boundary and flatters every arm equally. And **every state is emitted twice with the sides swapped**, because antisymmetry in the two players is a property of the game; a model trained on p1's view alone will not respect it.

Store raw, analyse on top. ABRA saves every game with every fact it may ever need. Every filter — rating tier, humans-only, archetype, playstyle — runs over the store at read time. A change to how the games are segmented is a re-computation, not a re-download. This makes the fetch a one-time cost and keeps the analysis free to change.

Support decisions, do not predict outcomes. In this format, the winner of a game is near-impossible to predict from the two team sheets; even a player-rating model ties a coin. ABRA therefore judges each model on a decision, not on the match result. Each probability ships a proper score (log-loss or Brier), a confidence interval, and an honest baseline.

Why the confidence interval is clustered. States within one game are correlated. A confidence interval that resamples states would be too narrow. ABRA resamples whole games instead. For a matchup matrix, it also resamples each cell from its Beta distribution before it solves, so the interval carries the small-sample uncertainty.

Honest negatives are kept. Two results are negative and are reported plainly: the team-picker's brings do not beat a coin (CHOMP-EV), and the playstyle rock-paper-scissors cycle rests on small samples and is suggestive, not settled. A negative that is measured is more useful than a positive that is asserted.

Companion documents. White paper · Deck · Project summary · Model ledger · Changelog